Deep Learning - COSC2779/2972

Deep Learning Hardware and Software

A/Prof. Ruwan Tennakoon

RMIT UNIVERSITY

Why Now?

Big Data Larger Data sets. Easier collection and storage.

IMAGENET

Computation Graphic Processing Units. Massively parallelizable.

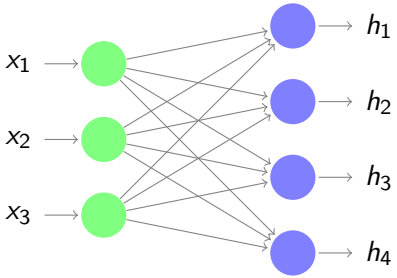


Software Improved Algorithms Widely available open source frameworks.

PyTorch

TensorFlow

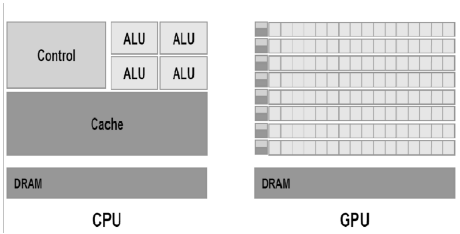
Most neural network operations can be represented as matrix manipulations.



$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

How can we do such operations faster?

A GPU is a specialized processor with dedicated memory that conventionally perform floating point operations required for rendering graphics



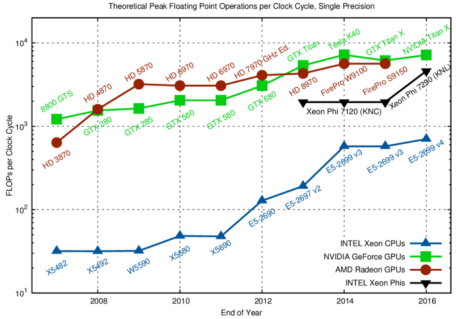


Image: Fast.ai

Good article on CPU vs GPU

Simple Comparison - CPU vs GPU

Simple matrix multiplication. Code run on Colab with Nvidia GPU. PyTorch handles device placement (`.to('cuda')`) explicitly.

Numpy (CPU):

```
import numpy as np
import timeit
A = np.random.rand(5000, 5000).astype(np.float32)
B = np.random.rand(5000, 5000).astype(np.float32)

timer = timeit.Timer("np.dot(A, B)", "import numpy as np; from __main__ import A, B")
print('Numpy mean time (s): ', np.mean(timer.repeat(10, 1)))
```

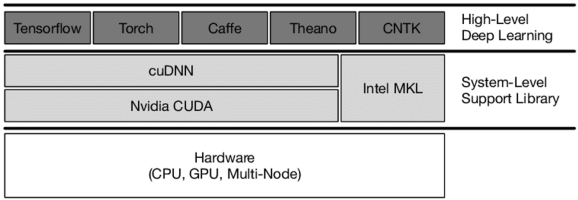
PyTorch (GPU):

```
import torch

# Move tensors to GPU
A_pt = torch.rand(5000, 5000).cuda()
B_pt = torch.rand(5000, 5000).cuda()

# Warm up and time
timer = timeit.Timer("torch.mm(A_pt, B_pt)",
                     setup="import torch; from __main__ import A_pt, B_pt")
print('PyTorch GPU mean time (s): ',  np.mean(timer.repeat(10, 1)))
```

Deep Learning Frameworks



- **PyTorch: Meta (Facebook), NYU**
- Tensorflow: Google
- JAX: Google Research
- MXNet: Amazon
- PaddlePaddle: Baidu

We will be using PyTorch for this course.

Why PyTorch?

- **Imperative Programming:** Uses "Eager Execution" by default. If you know Python, you know PyTorch.
- **Dynamic Computational Graphs:** The graph is built on-the-fly. Easy to debug using standard Python debuggers (PDB).
- **Research Standard:** Majority of top-tier AI conference papers (CVPR, NeurIPS) provide code in PyTorch.
- **Strong Ecosystem:** TorchVision, TorchText, and HuggingFace integration.

Quick Comparison: PyTorch vs TensorFlow in 2026

Metric	PyTorch	TensorFlow
Market Share	25.69%	37.51%
Companies Using	17,196	25,099
Research Papers	~85%	~15%
GitHub Stars	~81,000	~181,000
Job Postings	37.7%	32.9%
Primary Strength	Research, Flexibility	Production, Scale
Mobile/Edge	PyTorch Mobile	TensorFlow Lite (LiteRT)
High-Level API	PyTorch Lightning	Keras 3

How to Build a Simple Model



RMIT UNIVERSITY

In PyTorch, we define models by subclassing `nn.Module`.

- `__init__`: Define your layers here.
- `forward`: Define the data flow.

Alternatively, use `nn.Sequential` for simple stacks.

```python
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.main = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.main(x)

model = SimpleNet()
```

Training a Model (The PyTorch Way)

Unlike Keras' `model.fit()`, PyTorch requires an explicit training loop:

1. **Forward Pass:** Compute prediction.
2. **Loss:** Calculate error.
3. **Backward Pass:** `loss.backward()` calculates gradients.
4. **Optimizer Step:** `optimizer.step()` updates weights.

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()

for epoch in range(5):
    for images, labels in train_loader:
        # 1. Forward
        outputs = model(images)
        loss = criterion(outputs, labels)

        # 2. Backward
        optimizer.zero_grad() # Clear old gradients
        loss.backward()

        # 3. Update
        optimizer.step()

    print(f"Epoch {epoch} complete. Loss: {loss.item()}")
```